Documentation of Understanding

AI Agent Orchestration Platform — Yuno AI Engineer Assessment

Author: Shivansh Tripathi · Date: 31 May 2026 · Repository: see submission · Live demo: see submission

---

1. How I read the problem

The brief asks for a **platform to create, configure, and orchestrate AI agents** — not a single chatbot. Stripped to its essence, a strong submission must prove four things are *real*, not mocked:

- **Configurability** — define an agent's whole behavior (personality, model, tools, memory, schedule, guardrails, limits) and compose agents into workflows, from a UI.
- **A real runtime** — agents actually execute: call an LLM, run real tools in a loop, and hand work to each other to finish a task autonomously.
- **Reachability** — at least one agent is reachable by a human over an external messaging channel, conversationally.
- **Observability** — you can watch it happen (logs, inter-agent messages, token/cost), and history is persisted and visible.

Everything else (templates, tests, docs, single-command local run) serves to make those four credible and demonstrable. I treated the **grading weights** as the spec: *working end-to-end demo (40%)* and *architecture / code quality (30%)* dominate, so I optimized for a demo that genuinely runs and a codebase whose layering survives a code review.

The insight that shaped the design

A "visual workflow builder with conditions and feedback loops" and "a real runtime" are the *same object* if you choose the runtime well. So I made **the diagram the program**: the graph a user draws in the UI *is* the executable graph the runtime runs — no translation layer, nothing to drift. That single decision keeps the system honest.

---

2. Requirement → how it's satisfied

Requirement	Approach
Agent CRUD + rich config	`Agent` model with 12+ configurable dimensions; full editor in the UI

Real runtime executing agent logic	**LangGraph**: each node is a real ReAct agent (`create_react_agent`) running real tools
Visual builder w/ conditions + feedback loops	React Flow canvas → compiled 1:1 into a LangGraph `StateGraph`; conditional edges = conditions, back-edges = loops
≥ 2 workflow templates	*Content Pipeline* (Researcher→Writer→Editor with a `REVISE` feedback loop) and *Support Triage* (conditional routing to Billing/Tech)
Async agent-to-agent comms	In-process **EventBus**: agents emit messages; the engine routes — never direct calls
External channel	**Telegram** (long-polling, so no public webhook needed for local-first)
Persisted, visible history	One unified `Message` stream (chat / agent / tool / log / error), shown in Monitor + Chat
Live monitoring (logs, msgs, token/cost)	WebSocket `/ws/monitor`; cost from each model's real `usage_metadata`
Runs fully local, one command	`docker compose up --build` (multi-stage image serves UI from the backend)

3. Architecture, and why it's shaped this way

Three layers with a single contract between them — the separation the brief explicitly grades:

- **UI layer** — React SPA + FastAPI routers. The HTTP/WebSocket API is the *only* thing the UI knows about the system.
- **Agent runtime** — `app/runtime/` (LangGraph). Knows nothing about HTTP. Compiles stored workflow graphs into `StateGraph`s and executes ReAct agents with tools, memory, guardrails, and token/cost accounting.
- **Persistence** — SQLModel over SQLite. `DATABASE_URL` is the single switch to Postgres.

How a multi-agent task runs: the workflow's stored graph `{entry, nodes, edges}` compiles into a LangGraph `StateGraph`. Each node runs an agent as a genuine ReAct loop; each edge carries a condition (`always` / `contains:X` / `not_contains:X`). A back-edge (e.g. Editor → Writer while the reply contains `REVISE`) is a real feedback loop, bounded by a per-node visit cap + global recursion limit so it always terminates. Every hand-off is persisted *and* streamed to the monitor

through one code path — which is exactly what makes the agent-to-agent communication both **asynchronous** and **observable**.

---

4. Key decisions and trade-offs

Full set in `docs/adr/`. The load-bearing ones:

- **Runtime — LangGraph.** Its node / conditional-edge / cycle model maps 1:1 onto "visual builder with conditions and feedback loops," and `create_react_agent` gives a real tool-executing loop. CrewAI/AutoGen were weaker on first-class conditional branching; a custom runtime meant reinventing routing + recursion safety.
- **Multi-provider LLM, routed by model id.** Agents run on **Anthropic Claude** *or* **AWS Bedrock** (Amazon Nova / Llama); the provider is derived from the model id, so no provider/model drift. Verified live on both — directly serving "configurability."
- **Telegram via long-polling.** Lowest-friction "talk to your agent from your phone" with **no public webhook**, satisfying local-first. The `Channel` abstraction keeps Slack/WhatsApp a small addition; webhooks are the right move once hosted (a config switch).
- **SQLite + SQLModel.** Zero-setup local persistence; one env var to move to Postgres.
- **In-process EventBus.** One mechanism serves async comms *and* live monitoring; history and the live stream are the same data. Redis pub/sub for multi-node scale, behind the same interface.
- **Guardrails + cost in the runtime.** Limits (`max_iterations`, recursion cap) and guardrails (blocked keywords, per-run cost ceiling) are enforced where execution happens — the only place that can stop a runaway loop or spend.

---

5. What I would do next

- **Scheduler:** agents store a `schedule_cron`; wiring APScheduler to auto-trigger them is the next step.
- **Horizontal scale:** EventBus → Redis pub/sub, runs → a task queue, Telegram → webhooks behind a load balancer.
- **Auth & multi-tenancy:** OIDC + per-user scoping for a hosted product (omitted by design for a local-first single-user demo).
- **Richer guardrails:** PII filters and per-workflow tool allow-lists on the same hook.

---

6. How to evaluate this submission

- **Run it:** `cp .env.example .env` (add an LLM key — Bedrock or Anthropic), then `docker compose up --build` → `http://localhost:8000`. Seeded with 7 agents + 2 workflows.

- **See the multi-agent demo:** Workflows → *Content Pipeline* → Run; watch the Monitor as Researcher → Writer → Editor loops on `REVISE` then `APPROVED`, with live token/cost.
- **Talk to an agent:** message the Telegram bot; the conversation appears in the UI.
- **Read the reasoning:** `README.md`, `docs/adr/`, `docs/PRD.md`. Critical paths are covered by tests.
